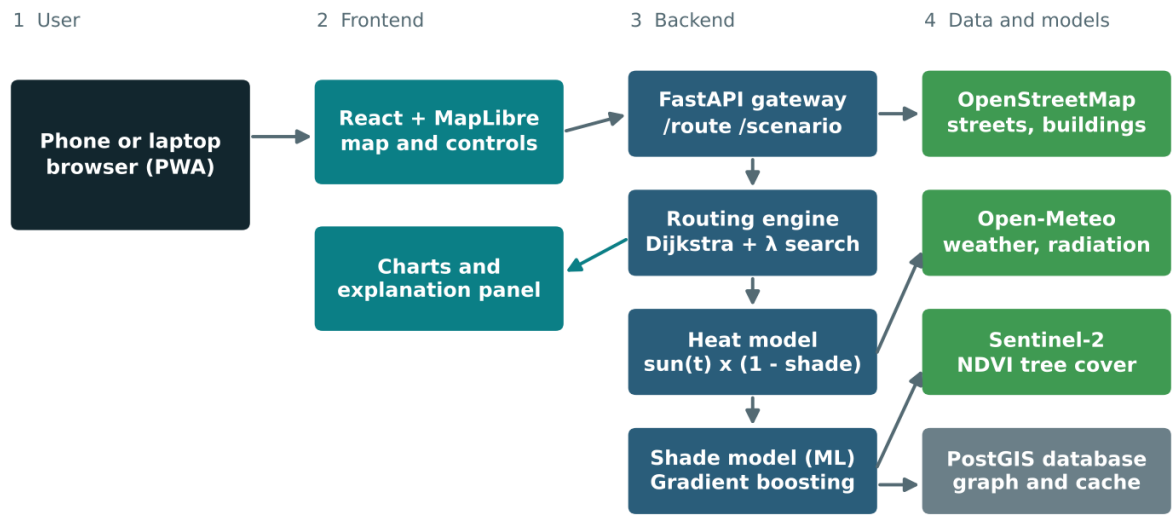
Backend and Architecture

Services, data model, algorithm, database and deployment

CoolRoute: Pedestrian Routing with Shade and Heat-Exposure Budgets

1. Architecture at a glance



Free-tier tools only. Prototype runs fully in the browser with synthetic data.

Figure 1. Four layers: user, frontend, backend, data and models.

2. Backend services

Service	Job	Status
API gateway	Checks input, returns JSON or clear errors	Built (Starlette; moves to FastAPI with small changes)
Routing engine	Shortest path and heat-budget path (Dijkstra + lambda search)	Built and tested
Heat model	heat = length x (1 - shade) x sun(t), plus crossing waits	Built and tested
Sun model	Sun strength from latitude, date and hour	Built and tested
Shade model (ML)	Predicts shade for each street piece from buildings and trees, with a confidence value	Demo trained on synthetic data
Data loader	Builds the street graph from OpenStreetMap, adds tree and building data	Plan (file 05)
Cache	Stores shade values by street piece and hour so routes are fast	Plan

3. Data model: one street piece (edge)

Field	Meaning	Example
length	Length in metres	82.4
shade	Share of the street in shade, 0 to 1	0.20
slope	Steepness in percent	3
step_free	True if no steps or high kerb	true
wait	Average wait at the crossing, seconds (0 if no crossing)	35
crossing_shade	Share of the crossing in shade	0.10
confidence	How much we trust the data, 0 to 1. Below 0.3 is shown as missing data	0.71
closed	True if the footpath is closed in this scenario	false

4. The algorithm in plain steps

```
function plan(graph, A, B, hour, budget_pct, step_free, shade_on):
    sun   = sun_strength(hour)
    adj   = build adjacency, skipping closed edges and (if step_free) non-step-free edges
    base  = dijkstra(adj, A, B, lambda = 0)                 # shortest distance
    if base is None: return NO_ROUTE
    budget = base.heat * budget_pct / 100
    if base.heat <= budget: return base, base
    lo, hi = 0, 50
    repeat 40 times:
        mid  = (lo + hi) / 2
        path = dijkstra(adj, A, B, lambda = mid)            # minimise distance + mid * heat
        if path.heat <= budget: best = path; hi = mid       # cool enough -> try a smaller lambda
        else:                   lo = mid                    # too hot -> push lambda up
    if nothing met the budget: best = dijkstra(..., lambda = hi)   # coolest available, flagged
    return base, best
```

Cost: each Dijkstra run is O(E log V). We run it at most 41 times, so a request is about 41 x O(E log V). For a city-size graph we would add contraction hierarchies or a bounding box around A and B so each run stays fast.

5. Database plan (production)

```
-- PostgreSQL with PostGIS
CREATE TABLE edges (
  id BIGSERIAL PRIMARY KEY, a BIGINT, b BIGINT, geom geometry(LineString, 4326),
  length_m REAL, slope_pct REAL, step_free BOOLEAN, wait_s REAL, crossing_shade REAL
);
CREATE TABLE edge_shade (
  edge_id BIGINT REFERENCES edges(id), hour SMALLINT, shade REAL, confidence REAL,
  PRIMARY KEY (edge_id, hour)
);
CREATE INDEX edges_geom_idx ON edges USING GIST (geom);
```

6. Privacy and safety

- No account is needed. Start and end points are used for the request and not stored.
- Heat exposure is a **comparison score**. We never state a personal health risk. Every API response carries a note saying so.
- Input is checked: bad ids, bad hours and bad budgets return a 400 error with a clear message (file 05).
- When data is missing we lower the confidence and warn the user. We do not hide the gap.

7. Deployment (free-tier plan)

Part	Option	Note
Frontend	GitHub Pages or Netlify	Static files. Check current free limits before relying on them
Backend	A free container host such as Render or Hugging Face Spaces	Free tiers may sleep when idle. Warm it up before the demo
Database	A free PostgreSQL tier with PostGIS, or SQLite for the hackathon	The prototype needs no database
Fallback	The browser prototype	Works with no backend and no internet, so the demo cannot fail

8. Backend tests

We start the server, call it over HTTP, and check 12 cases. All 12 passed (see file 07).

9. Run it

```
pip install uvicorn starlette
python3 -m uvicorn api_server:app --port 8765
curl -X POST localhost:8765/api/v1/route -H "Content-Type: application/json" \
     -d '{"origin":{"node":0},"destination":{"node":63},"hour":13,"heat_budget_pct":60}'
```